STAT 2102 · Practice Midterm: Answer Key

Every answer below was run in R; the lines starting with #> are R's actual output. Your numbers for the simulations match only if you use the same set.seed() and the same code order.

Problem 1: Functions

1.1 (5 pts)

Write a function named **roll_until_doubles** that takes no arguments. It should roll two fair six-sided dice repeatedly until both dice show the same number (doubles), and return how many rolls it took. Run your function once after `set.seed(1)`. Then use `replicate()` to estimate the average number of rolls over 10,000 runs. What value would you expect, and why?

```
roll_until_doubles <- function() {
  rolls <- 0
  repeat {
    rolls <- rolls + 1
    dice <- sample(1:6, 2, replace = TRUE)
    if (dice[1] == dice[2]) break
  }
  rolls
}
set.seed(1)
roll_until_doubles()
#> [1] 5
mean(replicate(10000, roll_until_doubles()))   # theory: 1 / (1/6) = 6
#> [1] 5.9949
```

Answer: One run took **5** rolls. The average over 10,000 runs is **5.99**, close to the expected **6**: P(doubles) = 6/36 = 1/6 on each roll, and the average wait for an event with probability p is 1/p. **Key ideas:** a counter starting at 0, `repeat` + `break` (or `while`), and `replace = TRUE` so both dice can show the same number.

On your site: Core 02 · Roll until something happens (while loop); Rec 4 Problems 1 and 5

1.2 (5 pts)

Write a function named **spin** with one argument, n, that simulates n spins of a spinner with these probabilities: Red 0.50, Blue 0.30, Green 0.15, Gold 0.05. The function should return a list containing (a) a frequency table of the outcomes that shows **all four colors**, even a color that never came up, and (b) the proportion of spins that landed on Red. Test it with `set.seed(2)` and `n = 12`.

```
spin <- function(n) {
  colors <- c("Red", "Blue", "Green", "Gold")
  spins <- sample(colors, n, replace = TRUE, prob = c(0.50, 0.30, 0.15, 0.05))
  list(counts = table(factor(spins, levels = colors)),
       prop_red = mean(spins == "Red"))
}
set.seed(2)
spin(12)
#> $counts
#>   Red  Blue Green  Gold
#>     5     4     3     0
#> $prop_red
#> [1] 0.4166667
```

Answer: Gold never came up in 12 spins, but it still appears with a 0 because of `factor(spins, levels = colors)`; plain `table(spins)` would leave it out. Red came up 5 of 12 times (0.417). The list is returned because the two outputs are different kinds of objects.

On your site: Core 01 · Unfair (weighted) dice; Core 05 · table() leaves out categories; HW 2 Problem 1 Part 2 practice

1.3 (5 pts)

Write a function named **letter_grade** that takes a numeric vector of scores and returns a vector of letter grades without using a loop: "A" for 90 and above, "B" for 80 up to (but not including) 90, "C" for 70 up to 80, and "F" below 70. Test it with `c(95, 83, 70, 64, 89.5)`.

```
letter_grade <- function(scores) {
  ifelse(scores >= 90, "A",
         ifelse(scores >= 80, "B",
                ifelse(scores >= 70, "C", "F")))
}
letter_grade(c(95, 83, 70, 64, 89.5))
#> [1] "A" "B" "C" "F" "B"
```

Answer: `"A" "B" "C" "F" "B"`. Watch the boundaries: 70 gets a C (70 or above), and 89.5 is a B, not rounded up to an A. Nested `ifelse()` labels the whole vector at once (or use `cut(scores, breaks = c(-Inf, 70, 80, 90, Inf), right = FALSE, labels = c("F", "C", "B", "A"))`).

On your site: Rec 3 Problem 1 Question 3 (tenure groups); Core 05 · if() on a whole vector → ifelse()

1.4 (5 pts)

Write a function named **mean_quartiles** with arguments `n`, `mu`, `sigma` and `reps` (default 1000). It should generate `reps` samples, each of `n` observations from a normal distribution with mean `mu` and standard deviation `sigma`, compute the mean of each sample, and return a **named vector** with the **first quartile, median, third quartile and interquartile range (IQR)** of the simulated sample means. Test it with `set.seed(3)`, `n = 25`, `mu = 70`, `sigma = 10`. Without running more code, how would you expect the IQR to change if `n` were 100 instead of 25?

```
mean_quartiles <- function(n, mu, sigma, reps = 1000) {
  means <- replicate(reps, mean(rnorm(n, mean = mu, sd = sigma)))
  q <- quantile(means, c(0.25, 0.50, 0.75))
  c(Q1 = q[[1]], Median = q[[2]], Q3 = q[[3]], IQR = q[[3]] - q[[1]])
}
set.seed(3)
mean_quartiles(n = 25, mu = 70, sigma = 10)
#>        Q1    Median        Q3       IQR
#> 68.524308 69.893487 71.220695  2.696387
```

Answer: Q1 ≈ 68.52, median ≈ 69.89, Q3 ≈ 71.22, IQR ≈ **2.70**. With n = 100 the IQR should be about **half** as large: the spread of sample means is σ/√n, so quadrupling n divides the spread by √4 = 2. (Running it with set.seed(3) and n = 100 gives an IQR of 1.40.) `q[[1]]` takes the plain number out of `quantile()`'s named result, so the names in the output are clean.

On your site: Core 03 · Sample means (CLT); Rec 3 Problem 1 Question 2 (quantile)

Problem 2: The coffee shop data

2.1 (1 pts)

Import `coffee_shops.csv` into R, converting all character variables into factors, and display the structure of the data.

```
coffee <- read.csv("coffee_shops.csv", stringsAsFactors = TRUE)
str(coffee)
#> 'data.frame':	400 obs. of  9 variables:
#>  $ Shop_ID  : int  1 2 3 4 5 6 7 8 9 10 ...
#>  $ Revenue  : num  36.2 23.7 37.7 35.1 56.4 36.1 30.5 45 36.1 12.2 ...
#>  $ Price    : num  5.89 4.47 4.6 4.92 4.51 4.21 5.8 3.83 5.12 4.42 ...
#>  $ Staff    : int  9 7 9 10 15 7 7 13 12 3 ...
#>  $ Age      : int  35 25 32 37 45 24 32 51 42 46 ...
#>  $ Location : Factor w/ 3 levels "Campus","Downtown",..: 2 1 3 2 2 1 2 3 2 3 ...
#>  $ DriveThru: Factor w/ 2 levels "No","Yes": 2 2 2 1 1 2 1 1 1 1 ...
#>  $ Chain    : Factor w/ 2 levels "No","Yes": 1 1 1 2 1 2 1 2 1 1 ...
#>  $ Rating   : Factor w/ 3 levels "Average","Excellent",..: 1 1 1 1 2 1 1 1 1 3 ...
```

Answer: 400 shops and 9 variables. Location, DriveThru, Chain and Rating are factors, which aggregate(), tapply(), table() and boxplot(y ~ g) need. Rating's levels are alphabetical (Average, Excellent, Poor) until you reorder them (Question 2.7).

On your site: Everyday code · Import data; HW 2 Problem 2 setup

2.2 (3 pts)

What is the average monthly revenue of the shops? How does the average revenue of chain shops compare to that of independent shops? Additionally, among **Campus** shops only, how does the average revenue of chain shops compare to that of independent shops?

```
mean(coffee$Revenue)
#> [1] 31.59325
aggregate(Revenue ~ Chain, data = coffee, FUN = mean)
#>   Chain  Revenue
#> 1    No 30.93575
#> 2   Yes 32.40503
campus <- coffee[coffee$Location == "Campus", ]
aggregate(Revenue ~ Chain, data = campus, FUN = mean)
#>   Chain  Revenue
#> 1    No 29.99400
#> 2   Yes 32.56667
```

Answer: The average shop makes **31.59** thousand USD a month. Chain shops average slightly more than independents (**32.41 vs 30.94**), and the gap is larger among Campus shops (**32.57 vs 29.99**).

On your site: HW 2 Problem 2 Part 1 (and its practice problem)

2.3 (2 pts)

Which shop charges the highest average drink price (give its index in the data set)? Conversely, which shop has the lowest revenue?

```
which.max(coffee$Price)
#> [1] 96
coffee[which.max(coffee$Price), c("Shop_ID", "Price", "Location", "Chain")]
#>    Shop_ID Price Location Chain
#> 96      96  6.58 Downtown    No
which.min(coffee$Revenue)
#> [1] 43
coffee[which.min(coffee$Revenue), c("Shop_ID", "Revenue", "Staff", "Location")]
#>    Shop_ID Revenue Staff Location
#> 43      43     7.5     3   Campus
```

Answer: Shop **96** charges the highest average price ($6.58; an independent Downtown shop). Shop **43** has the lowest revenue (7.5 thousand USD; a Campus shop with only 3 staff). `which.max()` / `which.min()` return the row index; use `which(x == max(x))` if ties are possible.

On your site: HW 2 Problem 2 Part 2

2.4 (3 pts)

What is the median revenue for each level of customer Rating? Answer the same question again, now also taking DriveThru into account, which gives six combinations.

```
aggregate(Revenue ~ Rating, data = coffee, FUN = median)
#>      Rating Revenue
#> 1   Average   33.75
#> 2 Excellent   45.05
#> 3      Poor   19.75
aggregate(Revenue ~ Rating + DriveThru, data = coffee, FUN = median)
#>      Rating DriveThru Revenue
#> 1   Average        No   32.90
#> 2 Excellent        No   44.10
#> 3      Poor        No   19.45
#> 4   Average       Yes   34.70
#> 5 Excellent       Yes   46.30
#> 6      Poor       Yes   21.35
tapply(coffee$Revenue, list(coffee$Rating, coffee$DriveThru), median)
#>              No   Yes
#> Average   32.90 34.70
#> Excellent 44.10 46.30
#> Poor      19.45 21.35
```

Answer: Median revenue rises with the rating: Poor **19.75**, Average **33.75**, Excellent **45.05**. Within every rating, shops with a drive-thru have a higher median (by about 1.8 to 2.2 thousand USD). The `tapply` grid shows the same six numbers as the `aggregate` data frame in an easier-to-compare layout.

On your site: HW 2 Problem 2 Part 3; Core 06 · Which summary tool?

2.5 (4 pts)

Quartiles. Find the first, second and third quartiles of Price. Compare the average revenue of shops priced **above the third quartile** with that of shops priced **below the first quartile**. Then add a new factor variable named `Price_Group` with levels Low (Price at or below Q1), Middle (above Q1, up to Q3) and High (above Q3), without modifying Price, and report how many shops fall in each group.

```
q <- quantile(coffee$Price, c(0.25, 0.50, 0.75))
q
#>    25%    50%    75%
#> 4.1100 4.4900 4.9425
mean(coffee$Revenue[coffee$Price > q[3]])     # most expensive quarter
#> [1] 30.063
mean(coffee$Revenue[coffee$Price < q[1]])     # cheapest quarter
#> [1] 34.01134
coffee$Price_Group <- cut(coffee$Price, breaks = c(-Inf, q[1], q[3], Inf),
                          labels = c("Low", "Middle", "High"))
table(coffee$Price_Group)
#>    Low Middle   High
#>    103    197    100
```

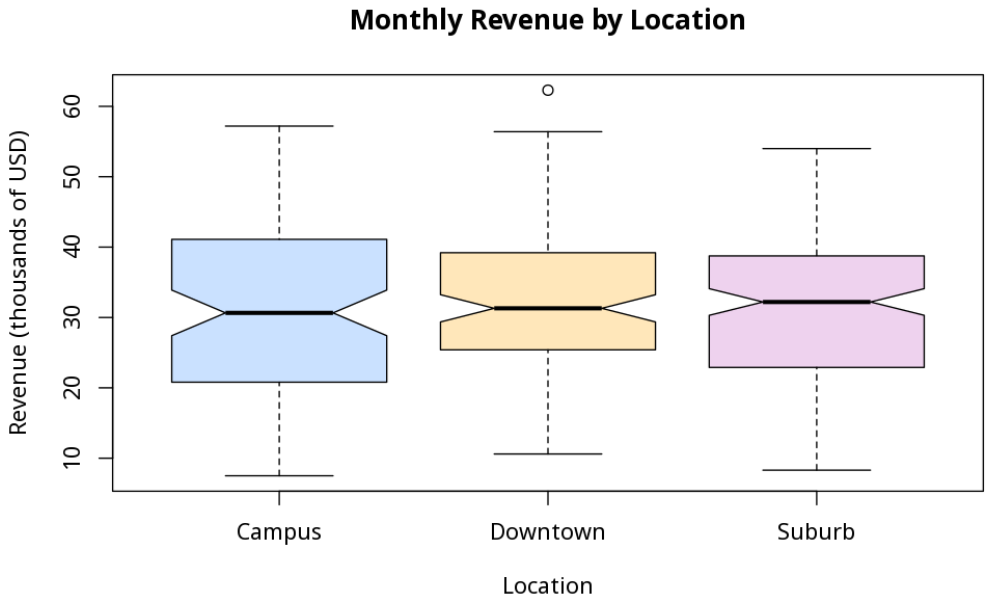
Answer: Quartiles of Price: Q1 = **4.11**, median = **4.49**, Q3 = **4.94**. Shops priced above Q3 average **30.06** in revenue, versus **34.01** for shops priced below Q1, so the most expensive quarter of shops earns about 4 thousand USD less. Groups: **Low 103, Middle 197, High 100**. Low has more than 100 because 6 shops are priced exactly at Q1, and "at or below Q1" counts them. `cut()` intervals include their right end, which matches "at or below" exactly.

On your site: Rec 3 Problem 1 Question 2 (25th / 75th percentiles); Rec 3 Question 3 and Lecture 3 (cut)

2.6 (3 pts)

Compare monthly revenue across the three locations using boxplots. Customize the boxplots by adding labels, changing the colors and adding notches. Are there any outliers?

```
b <- boxplot(Revenue ~ Location, data = coffee,
             notch = TRUE,
             col = c("lightsteelblue1", "wheat1", "thistle2"),
             main = "Monthly Revenue by Location",
             xlab = "Location", ylab = "Revenue (thousands of USD)")
b$out                                  # outliers (the dots)
#> [1] 62.3
```



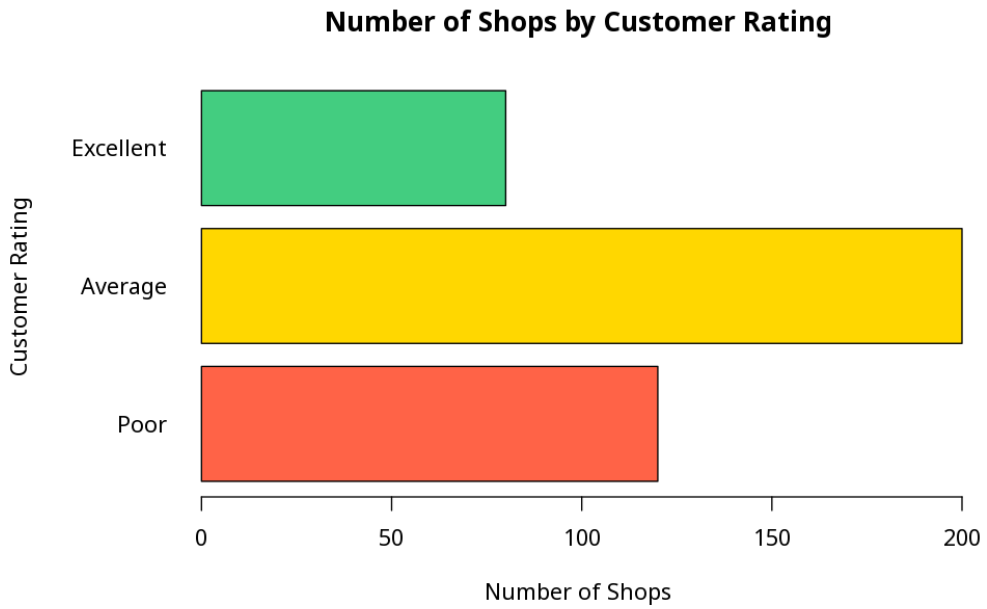
Answer: There is one outlier: a Downtown shop with revenue **62.3** (the dot above the Downtown box; `b$out` lists it). The medians are similar (Campus 30.65, Downtown 31.30, Suburb 32.20) and the notches overlap, so there is no clear difference in median revenue by location. Campus revenue is the most spread out.

On your site: HW 2 Problem 2 Part 4; Core 04 · Boxplot by group

2.7 (3 pts)

Provide an overview of customer ratings using a barplot. Display the bars in the order Poor, Average, Excellent, make the bars horizontal, and customize the plot with labels and colors.

```
coffee$Rating <- factor(coffee$Rating, levels = c("Poor", "Average", "Excellent"))
par(mar = c(5, 8, 4, 2))                # wider left margin so the bar names fit
barplot(table(coffee$Rating),
        horiz = TRUE,
        col = c("tomato", "gold", "seagreen3"),
        las = 1,                        # bar names written horizontally
        xlab = "Number of Shops",
        main = "Number of Shops by Customer Rating")
title(ylab = "Customer Rating", line = 6)
par(mar = c(5, 4, 4, 2))                # reset the margins
```



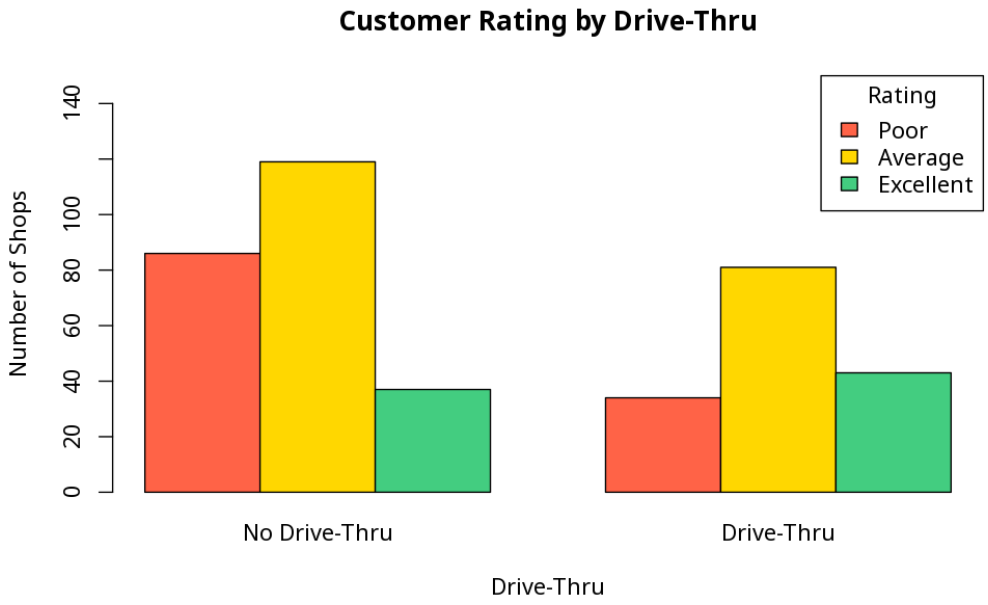
Answer: Average is by far the most common rating (200 shops), then Poor (120) and Excellent (80). Setting the factor levels puts the bars in the requested order. With `horiz = TRUE` the first level is at the bottom. **Tip:** long bar names get cut off on horizontal barplots, so widen the left margin with `par(mar = ...)` and reset it afterwards.

On your site: HW 2 Problem 2 Part 5; Core 04 · Barplot

2.8 (3 pts)

Create side-by-side barplots showing the frequency of each customer Rating for shops with and without a drive-thru. Include a legend, and customize the plot as in the previous question.

```
barplot(table(coffee$Rating, coffee$DriveThru),
        beside = TRUE,
        col = c("tomato", "gold", "seagreen3"),
        legend.text = TRUE,
        args.legend = list(x = "topright", title = "Rating"),
        names.arg = c("No Drive-Thru", "Drive-Thru"),
        ylim = c(0, 150),
        xlab = "Drive-Thru", ylab = "Number of Shops",
        main = "Customer Rating by Drive-Thru")
```



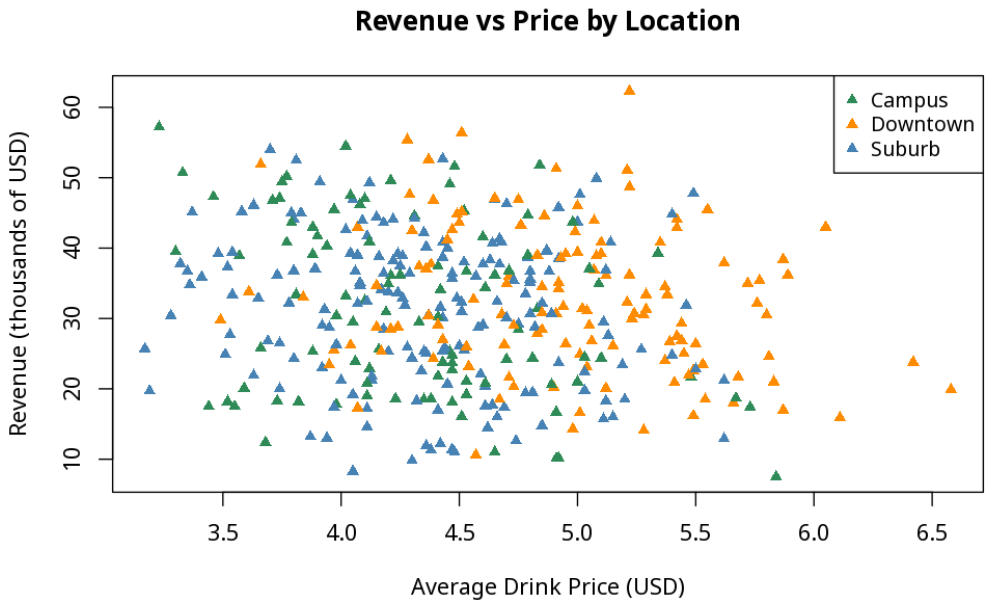
Answer: Shops with a drive-thru are more often rated Excellent: 43 of 158 (27%) versus 37 of 242 (15%) without one. Shops without a drive-thru are more often Poor (36% vs 22%). In `table(Rating, DriveThru)` the rows (ratings) become the bars and the columns become the groups.

On your site: HW 2 Problem 2 Part 6; Core 04 · Side-by-side barplot

2.9 (4 pts)

Plot Revenue against Price. Label the plot and axes properly. Use triangular points of size 0.9, colored by Location (one color for each of the three locations), and include a legend. Describe the relationship you see.

```
loc_cols <- c(Campus = "seagreen", Downtown = "darkorange", Suburb = "steelblue")
plot(coffee$Price, coffee$Revenue,
     pch = 17, cex = 0.9,
     col = loc_cols[as.character(coffee$Location)],
     xlab = "Average Drink Price (USD)", ylab = "Revenue (thousands of USD)",
     main = "Revenue vs Price by Location")
legend("topright", legend = names(loc_cols), col = loc_cols, pch = 17, cex = 0.9)
cor(coffee$Price, coffee$Revenue)
#> [1] -0.1514744
```



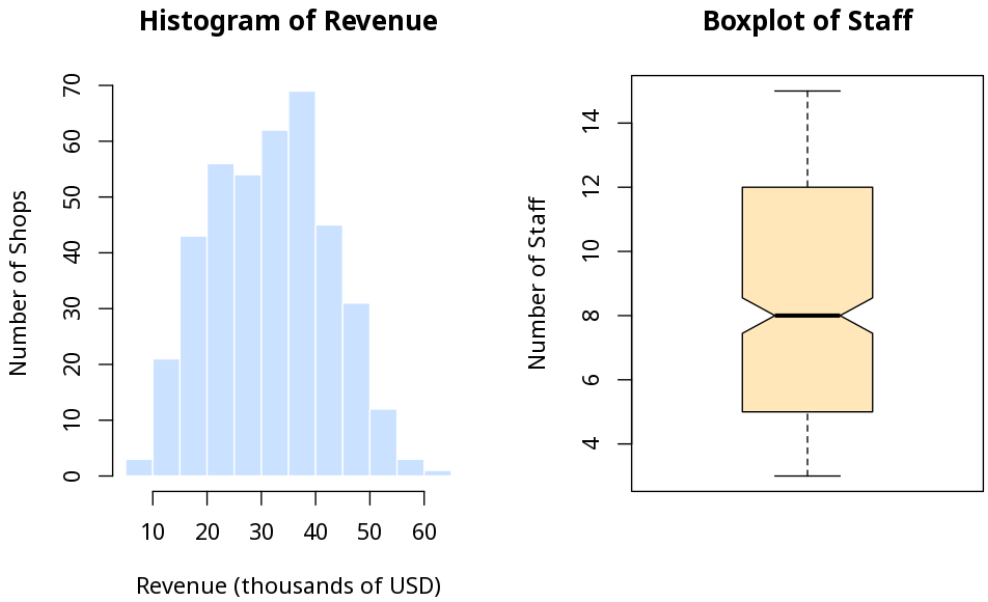
Answer: There is a weak negative relationship (correlation **-0.15**): more expensive shops tend to earn slightly less, but the points are widely scattered. Downtown shops (orange) charge the highest prices. Coloring through a named vector (`loc_cols[as.character(Location)]`) guarantees each location gets the color listed in the legend.

On your site: HW 2 Problem 2 Part 7; Core 04 · Scatterplot colored by group

2.10 (4 pts)

Combine a histogram of Revenue (about 10 bins) and a boxplot of Staff into a single image. Customize both plots as in Question 2.6 (no notch on the histogram).

```
par(mfrow = c(1, 2))
hist(coffee$Revenue, breaks = 10,
     col = "lightsteelblue1", border = "white",
     xlab = "Revenue (thousands of USD)", ylab = "Number of Shops",
     main = "Histogram of Revenue")
boxplot(coffee$Staff, col = "wheat1", notch = TRUE,
        ylab = "Number of Staff", main = "Boxplot of Staff")
par(mfrow = c(1, 1))
```



Answer: Revenue is roughly symmetric and bell-shaped, centered near 32 thousand USD (median 31.65). Staff ranges from 3 to 15 with a median of 8 and no outliers. `par(mfrow = c(1, 2))` puts the two plots side by side, and resetting to `c(1, 1)` afterwards keeps later plots full size.

On your site: HW 2 Problem 2 Part 8; Core 04 · Several plots in one window